T E C H N I C A L A R C H I T E C T U R E D O C U M E N T

Arquitectura Técnica del Sistema

Monolito Modular en Laravel con PostgreSQL — Implementación por Etapas

Proyecto: proyecto_cuentasxcobrar

Documento: ARQ-001 · Versión 1.1 (Laravel)

Stack: PHP 8.2 / Laravel 11 / PostgreSQL 16

Fecha: 11 de noviembre de 2026

Estado: Borrador para revisión

Confidencial — Uso interno

Índice de Contenido

Resumen Ejecutivo.....

Resumen Ejecutivo.....1

1. Introducción.....1

1.1 Propósito y alcance.....1

1.2 Relación con los documentos del proyecto.....2

1.3 Audiencia.....2

2. Visión Arquitectónica y Principios.....2

2.1 Estilo arquitectónico: monolito modular.....2

2.2 Principios de diseño.....3

2.3 Drivers de calidad.....4

3. Stack Tecnológico.....4

3.1 Visión general del stack.....4

3.2 Justificación del framework: Laravel 11.....4

3.3 Persistencia: Eloquent y Query Builder sobre PostgreSQL.....5

3.4 Cola y jobs: Laravel Queue + Horizon.....5

3.5 Frontend con Blade y Livewire.....5

4. Arquitectura en Capas.....5

4.1 Capas del sistema.....6

4.2 Reglas de dependencia y contenedor de servicios.....6

4.3 Estructura de directorios por módulo.....6

5. Módulos y Límites.....7

5.1 Catálogo de módulos (contextos delimitados).....7

5.2 Contratos entre módulos.....7

5.3 Propiedad de datos y un esquema por módulo.....7

6. Integración con el ERP.....8

6.1 Patrón conector: puerto y adaptador.....8

6.2 Carga batch nocturna....8
6.3 Consulta dirigida por API....8
6.4 Idempotencia y manejo de errores....9
6.5 Mapeo configurable de campos....9
7. Seguridad....9
7.1 Autenticación y gestión de sesiones....9
7.2 Autorización por roles (Gates y Policies)....9
7.3 Protección de secretos....10
7.4 Protecciones de aplicación....10
7.5 Auditoría de acciones....11
8. Observabilidad y Operación....11
8.1 Logging estructurado....11
8.2 Monitoreo y métricas....12
8.3 Alertas de falla de carga....12
8.4 Respaldos y recuperación....12
9. Estrategia de Pruebas....12
9.1 Pirámide de pruebas....12
9.2 Cobertura y calidad de código....13
9.3 Pruebas de carga y rendimiento....14
10. CI/CD y Despliegue....14
10.1 Pipeline de integración continua....14
10.2 Entornos....14
10.3 Despliegue y rollback....14
10.4 Estándares de código....15
11. Plan de Implementación por Etapas y Fases....15
11.1 Visión general....15
11.2 Detalle técnico por etapa....16
11.3 Fases internas de cada etapa....16
11.4 Criterios de paso entre etapas....17

12. Decisiones Técnicas Registradas (ADR)..... 17

13. Riesgos Técnicos y Mitigaciones..... 17

Nota: este índice se genera mediante códigos de campo. Tras editar el documento, haga clic derecho sobre el índice y seleccione “Actualizar campos” para refrescar los números de página.

Resumen Ejecutivo

Este documento define la arquitectura técnica del Sistema de Gestión de Cuentas por Cobrar: una aplicación web construida como monolito modular en PHP 8.2 con Laravel 11, persistencia en PostgreSQL 16 mediante Eloquent y el Query Builder, y módulos aislados con el paquete nwidart/laravel-modules. La arquitectura está diseñada para permitir la implementación de los requisitos por etapas y fases, de modo que cada etapa entregue un incremento operable del producto sin requerir reescrituras de lo construido. La integración con el ERP se materializa en un módulo conector aislado, con carga batch nocturna idempotente disparada por un comando Artisan y procesada por la cola de Laravel, y consulta dirigida por API; la operación transaccional se sostiene sobre una base de datos organizada en un esquema por módulo.

Las decisiones de diseño siguen los principios de separación de capas con inversión de dependencias resuelta por el contenedor de servicios de Laravel, límites de módulo estrictos con propiedad de datos, idempotencia en toda operación de carga apoyada en el UPSERT nativo de PostgreSQL, observabilidad mediante logging estructurado (Monolog) y auditoría por Observers, y seguridad por mínimo privilegio con control de acceso por Gates y Policies. El documento describe el stack tecnológico, la arquitectura en capas, los módulos y sus contratos, el patrón de integración con el ERP, la seguridad, la observabilidad, la estrategia de pruebas y el pipeline de CI/CD, y cierra con el plan de implementación por etapas y fases alineado con la Especificación de Requisitos.

1. Introducción

1.1 Propósito y alcance

El propósito de este documento es establecer la arquitectura técnica de referencia del sistema, en el nivel de detalle necesario para guiar el diseño detallado y la implementación de cada etapa. Cubre las decisiones de stack, la estructura en capas y módulos, los patrones de integración, la seguridad, la observabilidad, las pruebas, el despliegue y el plan por etapas. No entra en el detalle de cada pantalla ni en el diccionario de datos, que se tratan en sus documentos específicos (SRS y diseño de base de datos), pero establece las convenciones estructurales que ambos respetan.

1.2 Relación con los documentos del proyecto

La arquitectura se alinea con la Especificación de Requisitos (SRS v1.0), cuyas etapas y módulos determinan el orden de construcción, y con el Diseño de Base de Datos del módulo

de Integración (DDB-INT-001), cuyas tablas, restricciones de idempotencia y particionamiento se asumen como base de persistencia. La capa de acceso a datos accede a esas tablas mediante Eloquent y el Query Builder, y el versionado del esquema se gestiona con las migraciones nativas de Laravel, que ejecutan el DDL definido en el DDB. Los requisitos no funcionales del SRS —rendimiento, disponibilidad, seguridad, auditabilidad, mantenibilidad— son los drivers de calidad que esta arquitectura satisface y que se verifican en la Etapa 5 previa a la puesta en producción.

1.3 Audiencia

El documento está dirigido al equipo de desarrollo (líder técnico, desarrolladores PHP, DBA), al responsable de operaciones (despliegue, monitoreo, respaldos) y a la dirección del proyecto, que encontrará en el capítulo 11 el plan por etapas y los criterios de paso. Los lectores técnicos encontrarán las decisiones justificadas; los no técnicos, el contexto necesario en el resumen ejecutivo y en el plan por etapas.

2. Visión Arquitectónica y Principios

2.1 Estilo arquitectónico: monolito modular

El sistema se construye como un monolito modular: una única unidad desplegable cuya estructura interna respeta los límites de los módulos de negocio como si fueran servicios separados. Cada módulo agrupa su propia lógica de dominio, su capa de aplicación y su infraestructura, y es propietario de sus datos, que solo expone a los demás módulos mediante contratos explícitos (interfaces de dominio del módulo consumidor, nunca acceso directo a las tablas ajenas). En Laravel esta estructura se materializa con el paquete nwidart/laravel-modules, que provee un módulo con su propio ServiceProvider, sus rutas, sus migraciones y sus vistas. Esta decisión responde directamente a la restricción del proyecto: para el volumen de usuarios y la naturaleza de la operación, un monolito bien modularizado es más simple de construir, desplegar y operar que una arquitectura distribuida, y preserva la opción de extraer un módulo a servicio en el futuro sin reescritura.

2.2 Principios de diseño

- Separación de capas con inversión de dependencias: el dominio no depende de la infraestructura; esta implementa las interfaces que el dominio define y el contenedor de servicios de Laravel las resuelve.
- Límites de módulo estrictos: un módulo no accede a las tablas ni a las clases internas de otro módulo; solo a través de sus contratos públicos.

- Propiedad de datos: cada módulo es dueño de su esquema de base de datos; ningún otro módulo lo modifica directamente.
- Idempotencia por defecto: toda operación de carga o de cola puede re-ejecutarse sin efectos secundarios.
- Observabilidad integrada: toda operación relevante deja registro estructurado y trazable.
- Seguridad por mínimo privilegio: roles separados en base de datos y en aplicación; sin borrado físico desde la interfaz.
- Configuración sobre código: umbrales, reglas y catálogos son datos administrables, no constantes en el código.

2.3 Drivers de calidad

Los requisitos no funcionales del SRS son los drivers que la arquitectura debe satisfacer. El rendimiento interactivo (RNF-PER-01) se logra con consultas optimizadas e índices selectivos; la disponibilidad (RNF-DIS-01, RNF-DIS-02) con un despliegue sencillo y operación tolerante a fallas del ERP; la seguridad y la auditoría (RNF-SEG-01 a RNF-SEG-06) con el sistema de autenticación de Laravel, Gates y Policies, logging estructurado e inmutabilidad operativa; la mantenibilidad (RNF-MAN-01 a RNF-MAN-04) con la estructura modular, la configuración externalizada y el conector ERP aislado. La tabla 1 resume el mapeo.

Tabla 1. Drivers de calidad del SRS y respuesta arquitectónica

Driver (SRS)	Respuesta arquitectónica
Rendimiento interactivo (< 3 s)	Consultas con índices selectivos, caché de consultas frecuentes (Redis), paginación y eager loading para evitar N+1.
Carga batch (< 30 min)	Comando Artisan + cola de Laravel con UPSERT masivo idempotente en una transacción por entidad.
Disponibilidad y tolerancia a fallas	Despliegue monolítico simple; operación con última carga válida ante falla del ERP.
Seguridad y RBAC	Autenticación Laravel, Gates y Policies, roles de aplicación + roles BD, mínimo privilegio.
Auditabilidad e inmutabilidad	Logging estructurado (Monolog), Observers de auditoría, bitácora de integración, borrado lógico.
Mantenibilidad	Monolito modular con laravel-modules, conector ERP aislado, configuración en catálogos.
Escalabilidad sostenida	Particionamiento de tablas históricas; diseño que triplica el volumen sin cambio de arquitectura.

3. Stack Tecnológico

3.1 Visión general del stack

El stack se eligió buscando madurez, soporte a largo plazo, alineación con las buenas prácticas de la comunidad PHP y simplicidad operativa para un equipo pequeño. Todas las decisiones son de código abierto y se despliegan en un servidor dedicado o en contenedores. La tabla 2 resume el stack; las secciones siguientes justifican las decisiones principales.

Tabla 2. Stack tecnológico del sistema

Capa	Tecnología	Versión / nota
Lenguaje y runtime	PHP	8.2 o superior, con PHP-FPM.
Framework web	Laravel	11.x (cadencia de soporte anual, seguridad 2 años).
Estructura modular	nwidart/laravel-modules	Un módulo por contexto delimitado, con ServiceProvider propio.
ORM / acceso a datos	Eloquent + Query Builder	Eloquent para CRUD; Query Builder para UPSERT masivo idempotente.
Base de datos	PostgreSQL	16.
Cola y jobs	Laravel Queue + Horizon	Carga batch, notificaciones; Redis como transporte.
Plantillas	Blade	Renderizado del lado servidor.
Interactividad cliente	Livewire + Alpine.js	Reactividad sin build pesado; API JSON/Sanctum cuando se requiera.
Caché y colas	Redis	Transporte de cola y caché de consultas.
Servidor web	Nginx	Reverse proxy, TLS, estáticos.
Procesos	Supervisor + cron	Workers de cola y disparo de carga (Schedule).
Calidad de código	PHPStan, Pint, Pest/PHPUnit	Análisis estático, estilo PSR-12, pruebas.
Dependencias	Composer	Gestión de paquetes.

3.2 Justificación del framework: Laravel 11

Laravel se elige como framework por su curva suave, su ecosistema cohesionado y su ajuste natural al estilo de monolito modular cuando se complementa con nwidart/laravel-modules. Ofrece, integrados y consistentes, todos los componentes que el sistema necesita: un sistema de autenticación robusto, autorización por Gates y Policies, Eloquent como ORM activo con un Query Builder que ejecuta nativamente el UPSERT (ON CONFLICT) de PostgreSQL, una cola de jobs con reintentos y Observers de Horizon, el planificador Schedule para disparar la carga, Observers y eventos para la auditoría, y Blade con Livewire para una interfaz reactiva sin un pipeline de build frontend pesado. Su contenedor de servicios y el sistema de binding de interfaces a implementaciones permiten aplicar la inversión de dependencias sin acoplamientos mágicos, y la conformidad con los estándares PSR lo hace mantenible y portable. Frente a un enfoque más artesanal, Laravel acelera la entrega de la

base transversal (auth, colas, migraciones, validación) y deja al equipo concentrarse en las reglas de dominio.

3.3 Persistencia: Eloquent y Query Builder sobre PostgreSQL

Eloquent modela las entidades de dominio como modelos activos mapeados a las tablas de PostgreSQL definidas en el documento de base de datos, y se utiliza para las operaciones de lectura y de CRUD de la aplicación interactiva. Para las cargas masivas idempotentes desde el ERP se emplea el Query Builder y su método upsert(), que en PostgreSQL se traduce en INSERT … ON CONFLICT DO UPDATE, ejecutando el UPSERT de forma nativa y eficiente sin el overhead del ORM. Las migraciones nativas de Laravel gestionan el versionado del esquema; dado que el DDL del DDB es SQL estándar de PostgreSQL, las migraciones lo ejecutan mediante DB::statement, conservando las restricciones UNIQUE, los CHECK y el particionamiento declarativo. Las consultas de cartera y reportes se apoyan en las vistas de conveniencia definidas en la base de datos para evitar lógica de agregación duplicada, y se evita el problema N+1 con eager loading selectivo.

3.4 Cola y jobs: Laravel Queue + Horizon

Las operaciones asíncronas —carga batch, notificaciones de falla, reproceso, generación de reportes masivos— se implementan con Laravel Queue sobre Redis como transporte, supervisadas por Horizon para el monitoreo de throughput, reintentos y cuellos de botella. La carga batch nocturna es un comando Artisan (php artisan integracion:cargar) que encola jobs por entidad (clientes, facturas, servicios, pagos); los workers, bajo Supervisor, procesan el lote con reintentos controlados, backoff exponencial y registro de errores en la cola de fallidos (failed_jobs). El planificador Schedule (routes/console.php en Laravel 11) dispara el comando cada noche con withoutOverlapping para evitar ejecuciones solapadas. Este enfoque desacopla el disparo (Schedule) de la ejecución (worker) y aporta reintentos y observabilidad sin lógica ad hoc.

3.5 Frontend con Blade y Livewire

La interfaz se renderiza con Blade del lado del servidor, complementada con Livewire para la interactividad de las pantallas de gestión (filtros, modales, guardado de acciones, tablas reactivas) y Alpine.js para los comportamientos finos del cliente. Livewire es la opción nativa de Laravel para reactividad: mantiene el estado en el servidor y actualiza el DOM mediante peticiones ligeras, sin un pipeline de build frontend. Esta combinación mantiene al equipo dentro del ecosistema PHP y del estilo monolítico tradicional, adecuado a un equipo de desarrollo Laravel. Cuando un futuro módulo requiera una experiencia más rica (por ejemplo, un tablero interactivo de indicadores), se expondrán endpoints JSON consumidos por Inertia.js + Vue, sin alterar la arquitectura.

4. Arquitectura en Capas

4.1 Capas del sistema

Cada módulo implementa internamente cuatro capas con reglas de dependencia estrictas. La capa de presentación recibe las peticiones HTTP y devuelve respuestas; la capa de aplicación orquesta los casos de uso mediante Actions y Jobs; la capa de dominio contiene las entidades, value objects y reglas de negocio; y la capa de infraestructura implementa la persistencia con Eloquent y el Query Builder, los conectores externos y los workers de cola. La figura 1 muestra esta organización y los siete módulos que la replican verticalmente.

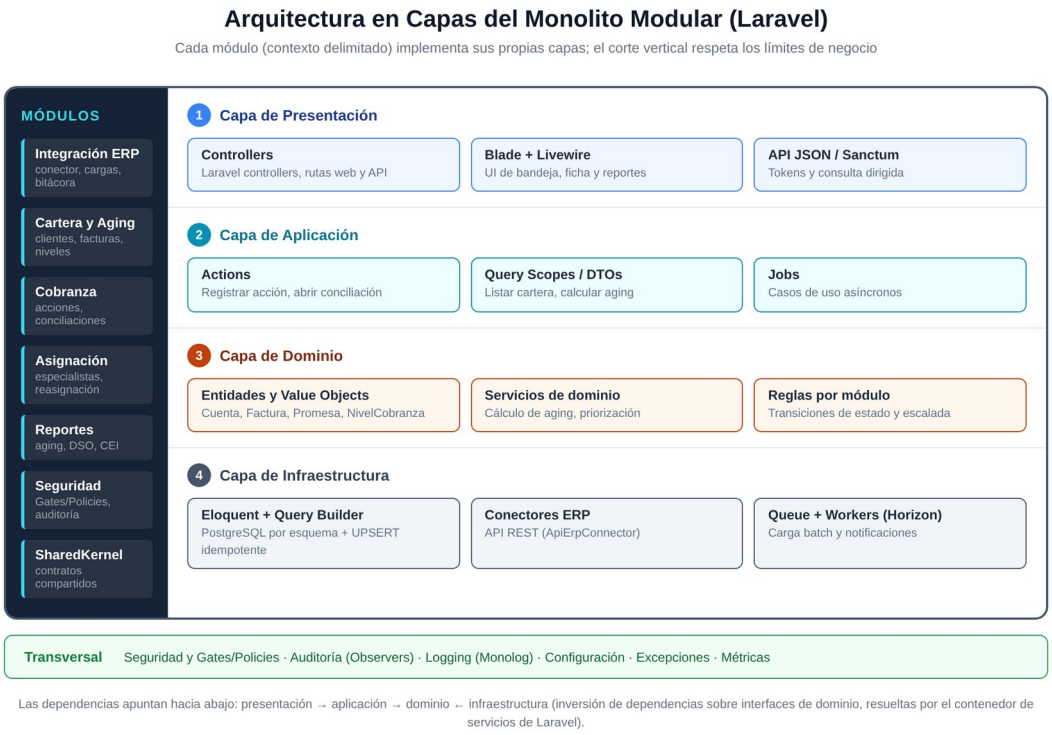


Figura 1. Arquitectura en capas del monolito modular (Laravel)

4.2 Reglas de dependencia y contenedor de servicios

Las dependencias apuntan hacia el dominio: la presentación depende de la aplicación, la aplicación del dominio, y la infraestructura implementa las interfaces que el dominio define (inversión de dependencias). En ningún caso el dominio importa clases de Eloquent, de Laravel framework o de otro módulo. El contenedor de servicios de Laravel resuelve los contratos: en el ServiceProvider de cada módulo se asocian las interfaces de dominio a sus implementaciones de infraestructura (por ejemplo, ErpSourceInterface a ApiErpConnector), de modo que el dominio declara qué necesita y la infraestructura queda conectable y reemplazable. Esta regla mantiene el dominio puro y verificable con pruebas unitarias sin

infraestructura, y permite cambiar la implementación de persistencia o el conector ERP sin tocar las reglas de negocio.

4.3 Estructura de directorios por módulo

Con nwidart/laravel-modules, cada módulo se materializa como un directorio bajo Modules/ con una estructura interna uniforme, su propio ServiceProvider, sus rutas, migraciones y vistas. Esta uniformidad facilita la navegación del código y el cumplimiento de los límites. El ejemplo muestra la estructura del módulo de Integración; los demás módulos la replican.

```
Modules/
└── Integracion/
    ├── Application/
    │   ├── Actions/        # EjecutarCargaAction, RegistrarErrorAction
    │   ├── Jobs/           # CargaBatchJob (Queue)
    │   └── DTO/
    ├── Domain/
    │   ├── Models/         # Carga, CargaDetalle, ErrorCarga (entidades de
dominio)
    │   ├── Services/       # CalculadorDeContadores, ValidadorDeFactura
    │   └── Contracts/      # ErpSourceInterface, CargaRepositoryInterface
    ├── Infrastructure/
    │   ├── Persistence/    # EloquentCargaRepository (implementa el contrato)
    │   ├── ErpConnector/   # ApiErpConnector, VistaBdErpConnector (adapter)
    │   └── Queue/          # CargaBatchWorker (worker de la cola)
    ├── Http/
    │   └── Controllers/    # CargaController (web + API)
    ├── Console/
    │   └── Commands/       # CargarCarteraCommand (Artisan)
    ├── Database/
    │   └── Migrations/     # Migraciones del módulo (Laravel)
    ├── Routes/             # web.php, api.php del módulo
    ├── Resources/
    │   └── views/          # plantillas Blade del módulo
    └── Providers/
        └── IntegracionServiceProvider.php  # binds de contracts
```

El módulo SharedKernel aloja los contratos y tipos compartidos (identificadores, value objects comunes como Moneda o FechaContable, y excepciones base) que varios módulos consumen sin violar límites, pues su contenido es estable y acordado. Se publica como un módulo más o como un paquete Composer interno.

5. Módulos y Límites

5.1 Catálogo de módulos (contextos delimitados)

El sistema se compone de siete módulos coincidentes con los del SRS. Cada módulo es un contexto delimitado con su propio lenguaje ubicuo, sus entidades y su esquema de base de datos. La tabla 3 resume su responsabilidad y su esquema.

Tabla 3. Módulos del sistema

Módulo	Responsabilidad	Esquema BD	Etapa
SharedKernel	Contratos y tipos compartidos estables.	—	0
Seguridad	Usuarios, roles, Gates/Policies, auditoría, configuración.	seguridad	0
Integración ERP	Conector, cargas, bitácora, errores, mapeo.	integracion	1
Cartera	Clientes, facturas, servicios, aging, niveles.	cartera	2
Cobranza	Acciones, conciliaciones, promesas, línea de tiempo.	cobranza	3
Asignación	Asignación y reasignación de clientes a especialistas.	asignacion	4
Reportes	Indicadores, aging, DSO, CEI, exportaciones.	reportes	5

5.2 Contratos entre módulos

Un módulo que necesita información de otro no accede a sus tablas ni a sus modelos Eloquent internos: invoca un contrato público definido como una interfaz del módulo proveedor, implementada en su capa de aplicación y resuelta por el contenedor de servicios. Por ejemplo, el módulo de Cobranza, al registrar una acción sobre una cuenta, consulta el saldo y el aging de la factura mediante FacturaQueryInterface del módulo Cartera, no leyendo directamente el modelo Factura ni la tabla factura. Este principio mantiene el acoplamiento bajo y localiza el impacto de los cambios. SharedKernel provee los tipos básicos que aparecen en los contratos (identificadores, Moneda, etc.) para evitar duplicar definiciones.

5.3 Propiedad de datos y un esquema por módulo

Cada módulo es propietario exclusivo de su esquema de base de datos y de las tablas que lo componen; ningún módulo escribe en el esquema de otro. Las lecturas cruzadas necesarias (por ejemplo, Reportes leyendo cartera y cobranza) se realizan mediante vistas de conveniencia o consultas explícitas a través del contrato del módulo propietario, nunca por acceso directo a sus tablas o modelos. Esta propiedad es la base de la futura extractibilidad: si un módulo se extrajera a un servicio, su esquema migraría con él sin afectar al resto.

6. Integración con el ERP

6.1 Patrón conector: puerto y adaptador

La integración con el ERP se aísla en el módulo Integración siguiendo el patrón de puerto y adaptador (hexagonal). El dominio del módulo define un puerto ErpSourceInterface con métodos como leerFacturas, leerServicios, leerPagos y leerClientes; la infraestructura proporciona adaptadores concretos, siendo ApiErpConnector —cliente del API REST del ERP— el adaptador implementado y configurado. El patrón puerto-adaptador preserva la opción de futuros adaptadores (vistas de base de datos o archivos), pero el mecanismo de acceso vigente y único en producción es el API REST. La selección se resuelve en el ServiceProvider del módulo según integracion.conexion_erp (tipo='api'), de modo que el dominio y el resto del sistema ignoran el mecanismo concreto y un eventual cambio no los afecta, satisfaciendo el requisito RNF-MAN-03.

```
// Puerto definido por el dominio del módulo Integración
namespace Modules\Integracion\Domain\Contracts;

interface ErpSourceInterface {
    public function leerClientes(\DateTimeImmutable $desde): iterable;
    public function leerFacturas(\DateTimeImmutable $desde): iterable;
    public function leerServicios(\DateTimeImmutable $desde): iterable;
    public function leerPagos(\DateTimeImmutable $desde): iterable;
}

// Adaptadores concretos (infraestructura) — ApiErpConnector (REST) es el
implementado
namespace Modules\Integracion\Infrastructure\ErpConnector;
final class ApiErpConnector     implements ErpSourceInterface { /* cliente
HTTP del API REST */ }
// final class VistaBdErpConnector implements ErpSourceInterface { /*
reservado */ }

// Binding en el ServiceProvider del módulo (selección por configuración)
namespace Modules\Integracion\Providers;
final class IntegracionServiceProvider extends ServiceProvider {
    public function register(): void {
        $this->app->bind(ErpSourceInterface::class, function ($app) {
            // tipo='api' (REST) es el valor configurado en
integracion.conexion_erp
            return match (config('integracion.conector')) {
                'api' => new ApiErpConnector(config('integracion.api')),   //
implementado
                default => new ApiErpConnector(config('integracion.api')),
            };
        });
    }
}
```

6.2 Carga batch nocturna

La carga batch se implementa como un comando Artisan (php artisan integracion:cargar) que crea un registro de carga en estado en_curso, encola en la cola de Laravel un job por entidad (clientes, facturas, servicios, pagos) y deja que los workers de Horizon lo procesen. Cada job lee los datos del ERP mediante el API REST (ApiErpConnector), valida cada registro, persiste con UPSERT idempotente mediante el método upsert() del Query Builder (que se traduce en INSERT … ON CONFLICT DO UPDATE) y registra los rechazos en la cola de

errores. Al terminar, actualiza el estado y los contadores de la carga y, si hubo errores o falla, dispara la notificación al administrador (RF-INT-06). La transacción por entidad, gestionada con DB::transaction, garantiza que cada conjunto se confirma entero o se revierte entero. El planificador Schedule dispara el comando cada noche y withoutOverlapping evita ejecuciones solapadas.

```
// Comando Artisan que dispara la carga batch
namespace Modules\Integracion\Console\Commands;
final class CargarCarteraCommand extends Command {
    protected $signature = 'integracion:cargar {--fecha= : Fecha de proceso,
por defecto hoy}';
    protected $description = 'Ejecuta la carga batch de cartera desde el ERP';

    public function handle(EjecutarCargaAction $action): int {
        $fecha = $this->option('fecha') ?? now()->toDateString();
        $action->execute($fecha);
        return self::SUCCESS;
    }
}

// Programación (Laravel 11): routes/console.php
use Illuminate\Support\Facades\Schedule;
Schedule::command('integracion:cargar')
    ->dailyAt('02:00')
    ->withoutOverlapping()
    ->onFailure(function () { /* notificar al administrador */ });

// UPSERT idempotente con Query Builder (PostgreSQL ON CONFLICT)
DB::table('cartera.factura')->upsert(
    $registros,                          // filas a insertar/actualizar
    ['clave_unicidad'],                  // columnas de la restricción UNIQUE

['saldo_pendiente','estado_contable','fecha_carga','carga_id','updated_at'] //
columnas a actualizar
);
```

6.3 Consulta dirigida por API

La consulta dirigida por API (RF-INT-07) permite refrescar el estado de una cuenta individual durante el día antes de una gestión comercial. Se expone como un endpoint interno invocado bajo demanda que crea una carga de origen api_dirigida, consulta el ERP para la factura y sus pagos, y actualiza el saldo y el estado contable mediante el mismo UPSERT idempotente. Esta consulta se realiza solo bajo solicitud explícita para no sobrecargar al ERP, y su ejecución queda registrada en la bitácora. Los endpoints se protegen con Sanctum cuando se consumen desde un cliente externo.

6.4 Idempotencia y manejo de errores

La idempotencia de la integración se garantiza en la base de datos mediante las restricciones UNIQUE sobre las claves naturales y la operación UPSERT (ON CONFLICT) ejecutada por el Query Builder, descritas en el documento de diseño de base de datos (sección 6.1). Los errores de validación de un registro (importe negativo, cliente inexistente, fecha incoherente)

se capturan por fila dentro de la transacción, se registran en integracion.error_carga con el dato original en JSON y no interrumpen la carga del resto del lote (RF-INT-04). Los jobs que fallan tras los reintentos configurados caen en la tabla failed_jobs de Laravel, desde donde el administrador puede inspeccionarlos y reprocesarlos. El reproceso controlado (RF-INT-08) reutiliza la misma lógica sobre un rango de fechas, dejando registro del usuario que lo disparó.

6.5 Mapeo configurable de campos

El mapeo entre los campos físicos del ERP y los campos del modelo local se externaliza en la tabla integracion.mapeo_campo_erp, de modo que un cambio en los campos del ERP se resuelve con un dato administrable y no con un cambio de código. El adaptador aplica el mapeo al leer los datos del ERP y, antes de persistirlos, el validador del dominio verifica que los campos marcados como obligatorios estén presentes y sean coherentes, rechazando los registros inválidos a la cola de errores.

7. Seguridad

7.1 Autenticación y gestión de sesiones

La autenticación se implementa con el sistema de autenticación de Laravel sobre dos mecanismos complementarios: el directorio corporativo LDAP como fuente principal de identidades y, para los usuarios que no figuren en él (cuentas técnicas, de administración o de servicio), usuarios propios gestionados en la base de datos local. La integración LDAP se materializa con el paquete LdapRecord-Laravel, que aporta un UserProvider propio para el guardia web: al iniciar sesión, el sistema valida las credenciales contra LDAP, sincroniza el usuario y sus atributos (nombre, correo, grupos) a la tabla local users y lo autentica; los usuarios propios se validan contra su hash local (bcrypt o argon2id). Ambas vías comparten el mismo guardia, las mismas Policies y la misma auditoría, de modo que el origen de la identidad es transparente para el resto del sistema. Las sesiones se gestionan con expiración por inactividad (30 minutos por defecto, parametrizable en config/session.php) y regeneración del identificador tras el login, en cumplimiento del requisito RNF-SEG-01. La integración futura con un proveedor de identidad corporativo (SSO vía SAML u OIDC) queda habilitada por la abstracción del guardia de Laravel (UserProvider), sin requerir reescritura del control de acceso.

7.2 Autorización por roles (Gates y Policies)

La autorización sigue el modelo de roles definido en el SRS (especialista comercial, supervisor, administrador de cartera, analista de conciliación, auditor, administrador

técnico). Laravel implementa los roles como middleware de ruta (role) y, para los accesos a recursos específicos, mediante Gates y Policies. Cada módulo define sus Policies: por ejemplo, CuentaPolicy verifica que un especialista solo vea y gestione las cuentas de los clientes que tiene asignados (RF-ASI-05), y CargaPolicy restringe el reproceso al administrador técnico. La matriz de permisos se documenta en el SRS y se materializa en Policies centralizadas por módulo, invocadas vía $this->authorize() en controladores y @can en Blade, de modo que la verificación es uniforme y auditable.

```
// Policy que verifica que el especialista solo gestione sus cuentas asignadas
namespace Modules\Asignacion\Policies;
final class CuentaPolicy {
    public function gestionar(User $user, Cliente $cliente): bool {
        if ($user->isSupervisor() || $user->isAdminCartera()) return true;
        return $user->especialista()->asignadoA($cliente->id)->exists();
    }
}

// Registro y uso
// app/Providers/AuthServiceProvider.php (o AuthServiceProvider del módulo)
protected $policies = [ Cliente::class => CuentaPolicy::class ];

// En el controller:  $this->authorize('gestionar', $cliente);
// En Blade:           @can('gestionar', $cliente) ... @endcan
```

7.3 Protección de secretos

Los secretos —credenciales del ERP, claves de la base de datos, parámetros sensibles— se gestionan como variables de entorno inyectadas en el despliegue y nunca se almacenan en el repositorio de código ni en archivos .env versionados. La tabla integracion.conexion_erp guarda únicamente una referencia al secreto (credenciales_ref), nunca el valor. Los logs se filtran para enmascarar campos sensibles antes de cualquier registro, configurando los atributos ocultos ($hidden) en los modelos y los canales de logging con procesadores de redacción, de modo que ninguna credencial llegue a la bitácora (RNF-SEG-05).

7.4 Protecciones de aplicación

- TLS obligatorio en todo el tráfico (HTTPS), con redirección de HTTP y HSTS.
- Cabeceras de seguridad: X-Content-Type-Options, X-Frame-Options, Content-Security-Policy básica (paquete o middleware propio).
- Protección CSRF en todos los formularios y operaciones de estado (@csrf en Blade); exclusión solo de rutas API con Sanctum.
- Validación de entrada con Form Requests (ValidateRequest) antes de cualquier escritura.
- Consultas parametrizadas vía Eloquent y Query Builder; prohibida la concatenación de SQL.

- Sin borrado físico desde la aplicación: borrado lógico (activo, fecha_baja) en todas las tablas transaccionales (RNF-SEG-06).
- Rate limiting en endpoints de API con throttle middleware.

7.5 Auditoría de acciones

Toda creación, modificación y eliminación lógica registra usuario, fecha, operación y valores anteriores y nuevos, en cumplimiento del requisito RNF-SEG-03. La auditoría se implementa con Observers de Eloquent registrados por módulo: un AuditoriaObserver escucha los eventos updating/updating/deleting de los modelos marcados como auditable y persiste un registro en la tabla auditoria del esquema seguridad mediante un listener asíncrono (cola), sin intervención del código de aplicación. La bitácora de integración complementa este registro con los eventos del proceso de carga. El auditor accede a ambas fuentes en solo lectura mediante el rol cxc_readonly.

```
// Observer de auditoría sobre entidades auditables
namespace Modules\Seguridad\Observers;
final class AuditoriaObserver {
    public function updating(Model $model): void {
        Auditoria::create([
            'entidad'          => get_class($model),
            'entidad_id'       => $model->id,
            'usuario_id'       => Auth::id(),
            'accion'           => 'update',
            'valores_anteriores' => json_encode($model->getOriginal()),
            'valores_nuevos'   => json_encode($model->getDirty()),
        ]);
    }
    // created, deleting ...
}

// Registro en el ServiceProvider del módulo (o AppServiceProvider)
Cliente::observe(AuditoriaObserver::class);
Factura::observe(AuditoriaObserver::class);
```

8. Observabilidad y Operación

8.1 Logging estructurado

El sistema registra su actividad con el Log facade de Laravel, respaldado en Monolog en formato estructurado (JSON), con canales separados por módulo y por propósito (app, integracion, security, deprecations) configurados en config/logging.php. Cada entrada incluye marca de tiempo, nivel, canal, contexto estructurado y un identificador de correlación por petición (request-id), inyectado por middleware, que permite seguir una operación a través de sus componentes. Los logs se enrutan a archivos rotativos (daily channel) en el servidor y, en producción, a un agregador central si la organización dispone de uno. Los niveles se configuran por entorno: debug en desarrollo, info en producción, con los errores y advertencias siempre presentes.

8.2 Monitoreo y métricas

La cola de jobs se supervisa con Laravel Horizon, que provee un tablero de throughput, tiempos de espera, reintentos y jobs fallidos. El sistema expone además un endpoint interno de métricas básicas (cargas del día, estado del último job, latencia de consultas críticas, tamaño de la cola) consumible por el sistema de monitoreo de la organización o por un dashboard operativo simple. La supervisión de cartera dispone, dentro de la propia aplicación, de un tablero de operación que muestra la última carga, los errores pendientes y las alertas activas. No se introduce un sistema de métricas dedicado adicional en la versión inicial; se aprovecha Horizon y el existente en la organización cuando lo haya.

8.3 Alertas de falla de carga

La falla de la carga nocturna o la finalización con errores dispara una notificación al administrador técnico y a la administración de cartera (RF-INT-06), por correo electrónico mediante las Notifications de Laravel y, si está configurado, por el canal de mensajería operativo. La aplicación continúa operando con la última carga válida, de modo que la falla no interrumpe la jornada. El reproceso queda a disposición del administrador desde la interfaz de administración, con registro del usuario y la causa.

8.4 Respaldos y recuperación

PostgreSQL se respalda diariamente con pg_dump completo y archivado continuo de WAL, lo que permite la recuperación a un punto en el tiempo dentro de las últimas 24 horas (RNF-DIS-03). El procedimiento de restauración se documenta y se prueba al menos una vez por ciclo en un entorno espejo. Las particiones mensuales de bitácora y snapshot se purgan por DETACH y DROP tras el período de retención, operación instantánea que no afecta la operación.

9. Estrategia de Pruebas

9.1 Pirámide de pruebas

La estrategia sigue la pirámide de pruebas: una base amplia de pruebas unitarias del dominio, una capa intermedia de pruebas de integración con base de datos de test, y un conjunto reducido de pruebas de extremo a extremo para los flujos críticos. Las pruebas se escriben con Pest (sintaxis expresiva sobre PHPUnit) o directamente con PHPUnit, apoyándose en el RefreshDatabase trait para las pruebas de integración, que recrean el esquema en una base de datos PostgreSQL de test por cada ejecución. Las pruebas unitarias del dominio son puras (sin framework, sin base de datos) y verifican las reglas de negocio (cálculo de aging, transiciones de estado, priorización). Las pruebas de integración verifican los repositorios,

los Actions y los flujos de carga idempotente. Las pruebas E2E cubren los flujos de carga, gestión comercial y asignación con usuarios de prueba.

9.2 Cobertura y calidad de código

- Cobertura objetivo del 70% en líneas y del 80% en ramas para el dominio, verificada con Pest/PHPUnit --coverage en el pipeline.
- Análisis estático con PHPStan en nivel 6+; sin errores nuevos en el pipeline.
- Estilo de código con Laravel Pint conforme a PSR-12, verificado en el pipeline.
- Pruebas de los adaptadores ERP con dobles (Mocks del puerto), sin depender del ERP real.
- Pruebas de idempotencia explícitas: dos ejecuciones del mismo job de carga producen los mismos datos.
- Factories de Eloquent para generar datos de prueba reproducibles por entidad.

9.3 Pruebas de carga y rendimiento

En la Etapa 5, previa a la puesta en producción, se ejecutan pruebas de carga al volumen proyectado (y a 3× el volumen para validar la escalabilidad, RNF-PER-05) sobre los flujos críticos: consulta de cartera con filtros, registro de acción y carga batch. Los objetivos son los del SRS: percentil 95 bajo 3 segundos en consultas interactivas y carga completa bajo 30 minutos. Los resultados se archivan como evidencia del cumplimiento de los requisitos no funcionales antes del go-live.

10. CI/CD y Despliegue

10.1 Pipeline de integración continua

Cada cambio enviado al repositorio dispara el pipeline de CI sobre GitHub Actions (o el sistema de CI de la organización). El pipeline ejecuta, en orden: instalación de dependencias con Composer, verificación de estilo con Laravel Pint, análisis estático con PHPStan, suite de pruebas unitarias y de integración con Pest/PHPUnit sobre una base de datos PostgreSQL de test efímera (servicio en el runner), y cálculo de cobertura. Un cambio se acepta solo si todas las etapas pasan. Las ramas se protegen y los cambios se revisan antes de fusionarse a la rama principal.

```
# Pipeline de CI (resumen de etapas, GitHub Actions)
jobs:
  quality:
    services:
      postgres: { image: postgres:16, env: { POSTGRES_*: ... } }
    steps:
```

```
      - composer install --prefer-dist --no-interaction
      - ./vendor/bin/pint --test                 # estilo PSR-12
      - ./vendor/bin/phpstan analyse --level=6   # análisis estático
      - php artisan key:generate
      - php artisan migrate --force              # sobre BD de test
      - ./vendor/bin/pest --coverage             # pruebas + cobertura
  build:
    needs: quality
    steps:
      - composer install --no-dev --optimize-autoloader
      - php artisan config:cache && php artisan route:cache
      - artifact: app.tar.gz
```

10.2 Entornos

Se disponen de cuatro entornos: desarrollo (local del desarrollador, con base de datos y ERP simulado), pruebas (compartido, con datos de prueba anonimizados de la cartera real), staging (espejo de producción, sobre el que se ejecutan las pruebas de aceptación y de carga) y producción. La configuración de cada entorno se entrega por variables de entorno; ningún secreto se versiona. Los datos de cartera de prueba se anonimizan antes de cargarse en los entornos no productivos.

10.3 Despliegue y rollback

La topología de despliegue, mostrada en la figura 3, consta de un servidor de aplicación con Nginx como reverse proxy, PHP-FPM ejecutando la aplicación Laravel, PostgreSQL con los esquemas por módulo y un planificador (cron + Supervisor) que dispara la carga batch y los workers de cola. El ERP contable es un sistema externo al que el conector accede por el mecanismo configurado (API, vistas de base de datos o archivos). Redis actúa como transporte de la cola y caché de consultas frecuentes, supervisado por Horizon.

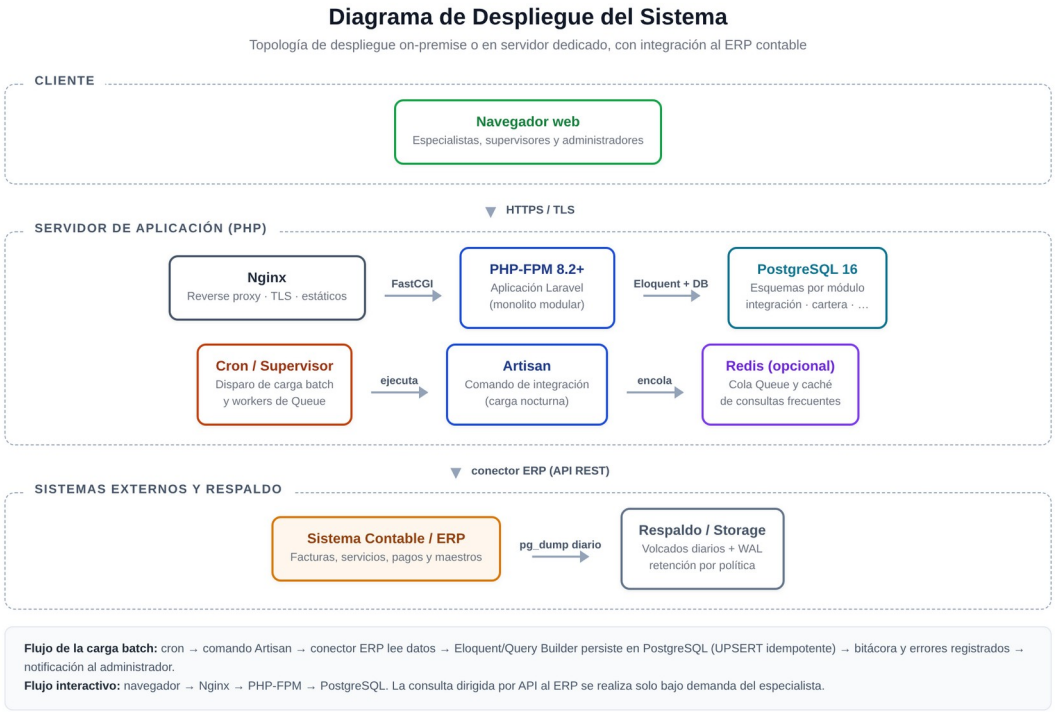


Figura 3. Diagrama de despliegue del sistema

El despliegue se realiza desde el artefacto producido por CI: se extrae el paquete en el servidor, se ejecutan las migraciones de Laravel (php artisan migrate --force, idempotentes y compatibles hacia atrás, sección 8.3 del DDB) y se reinicia PHP-FPM y los workers de cola bajo Supervisor (php artisan queue:restart). El despliegue incluye una verificación de salud (health check) que confirma la conectividad con PostgreSQL y con el ERP antes de declararlo exitoso. El rollback consiste en replegar el artefacto anterior y, solo si la migración lo requiere, revertirla con migrate:rollback; por la política de compatibilidad hacia atrás de las migraciones, la versión inmediatamente anterior opera sin cambios sobre el esquema actualizado.

10.4 Estándares de código

El código se adhiere a PSR-12 para estilo (verificado con Laravel Pint) y a PSR-4 para autoloading, con nombres de clases en PascalCase y nombres de métodos y variables en camelCase, conforme al idioma universal de la comunidad PHP y de Laravel. La documentación de las clases y métodos públicos se redacta en español con bloques PHPDoc, y los identificadores del dominio usan el lenguaje ubicuo de cada módulo (Carga, Factura, PromesaDePago, NivelCobranza). El repositorio mantiene la documentación (SRS, diseño de BD, este documento) sincronizada con el código, bajo la carpeta docs/.

11. Plan de Implementación por Etapas y Fases

11.1 Visión general

La implementación se organiza en seis etapas coincidentes con los módulos del SRS, cada una entregando un incremento operable del producto. Dentro de cada etapa, el trabajo se organiza en fases internas que siguen el ciclo diseño → desarrollo → pruebas → aceptación de usuario (UAT), de modo que cada etapa cierra con una entrega validada. La figura 2 muestra el roadmap completo con las dependencias entre etapas.



Figura 2. Roadmap de implementación por etapas y fases

11.2 Detalle técnico por etapa

La tabla 4 detalla, para cada etapa, los módulos involucrados, los entregables técnicos y la dependencia con etapas previas. Las duraciones son estimaciones de referencia para un equipo pequeño y se ajustarán en el plan de proyecto.

Tabla 4. Detalle técnico por etapa

Etapa	Módulos / entregables técnicos	Dependencia
Etapa 0 — Base	Andamiaje Laravel + laravel-modules; módulo Seguridad (auth, Gates/Policies, auditoría); esquemas y migración inicial; pipeline CI; entorno de pruebas.	Ninguna
Etapa 1 — Integración	Módulo Integración: puerto ErpSourceInterface, adaptador al ERP real, comando Artisan de carga con Queue/Horizon, UPSERT idempotente, bitácora y cola de errores, consulta dirigida por API.	Etapa 0
Etapa 2 — Cartera	Módulo Cartera: consulta y filtros, cálculo de aging, estados y niveles N0-N4, bandeja priorizada, ficha de cliente; enriquece el esquema cartera.	Etapa 1

Etapa	Módulos / entregables técnicos	Dependencia
Etapa 3 — Cobranza	Módulo Cobranza: registro de acciones, línea de tiempo, conciliaciones con ciclo de vida, promesas de pago, aprobaciones; esquema cobranza.	Etapa 2
Etapa 4 — Asignación	Módulo Asignación: asignación y reasignación con historial, masivas, visibilidad por cartera, carga de trabajo; esquema asignacion.	Etapa 2
Etapa 5 — Reportes y endurecimiento	Módulo Reportes: aging, DSO, CEI, exportaciones; pruebas de carga al volumen; revisión de seguridad; puesta en producción.	Etapas 3 y 4

11.3 Fases internas de cada etapa

Cada etapa se ejecuta en cuatro fases internas con entregables y verificaciones propios. Esta cadencia hace que cada etapa cierre con una entrega validada y que los riesgos se detecten temprano.

Tabla 5. Fases internas por etapa

Fase	Actividad	Entregable / verificación
F.x.1 Diseño	Diseño detallado del módulo: entidades, contratos, pantallas, migraciones.	Diseño revisado y aprobado.
F.x.2 Desarrollo	Implementación por capas con pruebas unitarias del dominio (Pest).	Código en rama con CI verde.
F.x.3 Pruebas	Pruebas de integración y de los flujos críticos; corrección de defectos.	Suite de pruebas en verde.
F.x.4 Aceptación	UAT con usuarios reales sobre datos anonimizados; revisión de trazabilidad.	Acta de aceptación de la etapa.

11.4 Criterios de paso entre etapas

Una etapa se considera cerrada y habilita la siguiente cuando se cumplen, de manera demostrable, las siguientes condiciones: la suite de pruebas unitarias e de integración pasa en el pipeline con la cobertura objetivo; los criterios de aceptación definidos en el SRS para los requisitos de la etapa se verifican positivamente en el UAT; la trazabilidad requisito → implementación → prueba está completa y revisada; y el entorno de staging opera de forma estable con la nueva funcionalidad integrada. Solo en la Etapa 5 se adicionan las pruebas de carga y la revisión de seguridad como condición de la puesta en producción.

12. Decisiones Técnicas Registradas (ADR)

Las decisiones arquitectónicas significativas se registran como ADR (Architecture Decision Records) en el repositorio, bajo docs/adr/. Cada ADR documenta el contexto, la decisión, sus

alternativas y sus consecuencias. La tabla 6 resume los ADR iniciales; el detalle de cada uno vive en su archivo correspondiente y evoluciona con el proyecto.

Tabla 6. Decisiones técnicas registradas

ADR	Decisión	Justificación resumida
ADR-001	Monolito modular frente a microservicios.	Simplicidad de construcción y operación para el volumen previsto; preserva la extractibilidad futura.
ADR-002	Laravel 11 como framework (frente a Symfony).	Ecosistema cohesionado (auth, Eloquent, Queue, Blade, Livewire); curva suave; módulos via laravel-modules.
ADR-003	PostgreSQL 16 como único gestor.	Idempotencia con UPSERT (ON CONFLICT), particionamiento declarativo, JSONB para contexto de errores.
ADR-004	Puerto y adaptador; adaptador API REST implementado.	Aislar el acceso al ERP; ApiErpConnector (REST) como mecanismo vigente, patrón preserva futuros adaptadores.
ADR-005	Eloquent + Query Builder (upsert() nativo para cargas).	Eloquent para CRUD; Query Builder para cargas masivas idempotentes.
ADR-006	Laravel Queue + Horizon para jobs.	Reintentos, backoff y observabilidad; transporte Redis.
ADR-007	Borrado lógico e inmutabilidad operativa.	Cumplir trazabilidad e inmutabilidad (RNF-SEG-03/06).
ADR-008	Blade + Livewire + Alpine para el frontend.	Reactividad nativa de Laravel sin build pesado; futuro Inertia+Vue si se requiere.
ADR-009	Migraciones de Laravel compatibles hacia atrás.	Despliegues sin tiempo de incautación y rollbacks seguros.
ADR-010	CI en GitHub Actions con Pint, PHPStan y Pest.	Calidad de código garantizada antes de fusionar.
ADR-011	Auditoría por Observers de Eloquent.	Trazabilidad automática sin acoplar el código de aplicación.
ADR-012	Autenticación LDAP (LdapRecord) + usuarios propios.	Directorio corporativo como fuente principal; usuarios locales para cuentas técnicas; un solo guardia y Policies.

13. Riesgos Técnicos y Mitigaciones

La tabla 7 identifica los riesgos técnicos más relevantes de la arquitectura y las mitigaciones previstas. Los riesgos se revisan al inicio de cada etapa y se actualizan conforme se conoce más del sistema contable de origen y de los datos reales de la cartera.

Tabla 7. Riesgos técnicos y mitigaciones

Riesgo	Impacto	Mitigación
El ERP no expone API y solo permite acceso por vistas/archivos.	Medio	El patrón puerto-adaptador aísla el mecanismo; se implementa el adaptador que corresponda sin tocar el dominio.
Variabilidad de los campos del ERP entre versiones.	Medio	Mapeo configurable en integracion.mapeo_campo_erp; el adaptador aplica el mapeo en runtime.
Crecimiento de tablas históricas (bitácora, snapshot).	Bajo	Particionamiento mensual y purga por partición; consultas con pruning.
Acoplamiento accidental entre módulos.	Medio	Límites por módulo laravel-modules, contratos por interfaz, revisión de código y análisis estático que prohíbe imports cruzados.
Falla del ERP durante la carga.	Medio	Operación con última carga válida; notificación al administrador; reproceso controlado; jobs fallidos en failed_jobs.
Pérdida de datos por falla de la base de datos.	Alto	Respaldos diarios + WAL; prueba periódica de restauración; replicación si la política lo exige.
Brecha de seguridad por credenciales expuestas.	Alto	Secretos por entorno; conexion_erp solo referencia; logs redactados; revisión de seguridad en la Etapa 5.
Curva de aprendizaje del equipo con Laravel/Eloquent.	Bajo	Estructura uniforme por módulo; documentación interna; pair programming en la Etapa 0.
Acoplamiento a patrones mágicos de Laravel.	Bajo	Mantener el dominio puro (sin facades ni Eloquent); bindings explícitos en ServiceProviders; análisis estático.

Este documento, junto con el SRS y el diseño de base de datos del módulo de Integración, constituye la base técnica del proyecto. Las decisiones pendientes (mecanismo concreto de acceso al ERP, proveedor de CI y de monitoreo de la organización, política de retención de auditoría) se completarán al inicio de la Etapa 0 y se registrarán como ADR en el repositorio.